OMNIX-RTE-001

Runtime Treasury Execution Trace ? v1.1.0

USD 50,000,000 | SWIFT MT202 / XRPL | Dilithium-3 ML-DSA-65 FIPS 204

WHAT THIS PACKAGE CONTAINS

A cryptographically verifiable dual-path execution trace for a $50,000,000 autonomous cross-border liquidity release. Two paths, same governance stack, same PQC keypair:

PATH DANGEROUS

Treasury agent with degraded authority attempted the transfer. Governance halted it. Hard refusal issued. Settlement blocked. No execution occurred. Proven cryptographically.

PATH ADMISSIBLE

Same agent, recertified under valid authority, executed the same transfer. Settlement released ? USD 50,000,000 via SWIFT MT202 and XRPL. Proven cryptographically.

VERIFICATION ? UNDER 2 MINUTES

Requirements: Python 3.9+

QUICK START

```
pip install pqc          # Dilithium-3 ? one-time, ~5 seconds
python verify.py         # runs all 111 checks
```

EXPECTED VERDICT

```
TOTAL CHECKS :              111
PASSED       :              111
FAILED       :              0
VERDICT      :              ALL VERIFICATIONS PASS
```

Without pip install pqc: the 26 PQC signature checks are SKIPPED (not FAILED). All 75 structural and hash integrity checks run and pass regardless.

ANCHOR ARTIFACT ? PROOF OF GOVERNANCE CERTIFICATE

Embedded in the package. PQC-signed. Verifiable without network access or OMNIX infrastructure.

POGC

```
ID:          POGC-F1DC0218E5204875
Tier:        MANDATE-BOUND
PQC:         ML-DSA-65 (Dilithium-3, FIPS 204)
Settlement:  USD 50,000,000
SWIFT ref:   MT202-2CAA8C200950
XRPL tx:     XRPL-8F7967D6A3A04ECC8C27CD0C
```

PACKAGE CONTENTS

```
OMNIX-RTE-001-REVIEWER/
```

```
verify.py                 standalone verifier (Python 3.9+)
README_FIRST.md           this overview
VERIFY.md                 all commands and expected outputs
QUICKSTART.md             step-by-step for first-time reviewers
evidence/
  OMNIX-RTE-001_*.json    complete evidence package (194 KB)
  OMNIX-RTE-001_*.pdf     institutional document (15 pages)
```

TARGETED VERIFICATION MODES

All commands run from the package root directory after unzipping.

1. Full verification (111 checks)

```
python verify.py
```

Expected: 111 / 111 PASS

2. Authority chain ? DR + MBR (both paths)

```
python verify.py --verify-authority
```

Expected: 15 / 15 PASS | Delegation Receipt integrity, MAR constraint, MIVP thresholds

3. Runtime continuity ? RCR + CES + MAS

```
python verify.py --verify-continuity
```

Expected: 17 / 17 PASS | CES formula T*0.30+B*0.30+D*0.20+I*0.20, band classification

4. Counterfactual engine ? CGE CFRs + CAT

```
python verify.py --verify-counterfactual
```

Expected: 17 / 17 PASS | 5 CFRs per path, CAT root hash, fragility scores

5. Dangerous path HALT enforcement

```
python verify.py --verify-halt
```

Expected: 25 / 25 PASS | HARD_REFUSAL, BLOCKED, OSG REJECTED, execution_occurred=False

6. Admissible path settlement

```
python verify.py --verify-settlement
```

Expected: 33 / 33 PASS | PoGC MANDATE-BOUND, OSG APPROVED, amount=50000000

7. Post-execution replay proofs

```
python verify.py --verify-replay
```

Expected: 19 / 19 PASS | TCS snapshots, Replay Proof hashes, terminal status

8. Machine-readable output

```
python verify.py --json
```

Expected: Structured JSON report for audit pipeline integration

---

WHAT THE VERIFIER DOES NOT CHECK

The verifier is scoped to cryptographic and structural integrity only. It does NOT validate governance policy values (FX bands, counterparty lists). It does NOT access the OMNIX platform, network, or any external service. It does NOT require the OMNIX private key ? only the embedded public key.

If all 111 checks pass: the package has not been altered since generation, all PQC signatures are valid, and the structural invariants are satisfied.

INSPECT THE POGC DIRECTLY

PYTHON

```
import json
pkg  = json.load(open("evidence/OMNIX-RTE-001_*.json"))
pogc = pkg["paths"]["path_admissible"]["steps"]\
            ["6_gate"]["proof_of_governance_certificate"]
print(json.dumps(pogc, indent=2))
```